

Module 5 — Read 2

Inference for slope, mean-response CI, and prediction intervals

Module 5, Read 2: Inference in simple linear regression

Project: [Project 5 — Looking Ahead](#) · **Prerequisite:** [Read 1](#) and [Quiz A](#) · **Next:** [Quiz B](#), [demo lab](#), [exercise lab](#).

Complete Read 1 and Quiz A first. This reading adds **interval estimates** for the slope, the mean response, and individual predictions — and explains why prediction intervals are always wider.

Learning outcomes

One section — use **Check yourself** before [Quiz B](#).

After this reading you should be able to:

Section 1: Inference in simple linear regression

Slope inference, mean-response CI, and prediction intervals

- State the residual-normality assumption that supports interval inference in linear regression.
- Interpret a **95% confidence interval for the slope** β_1 in context.
- Interpret a **confidence interval for the mean response** at a specific x_0 value.
- Interpret a **prediction interval** for an individual future response at x_0 .
- Explain why **prediction intervals** are wider than confidence intervals for the mean response.
- Use `confint()` and `predict(..., interval = "prediction")` in R to extract these intervals.

Section 1: Inference in simple linear regression

Inference in simple linear regression

Bridge from Read 1

You can already fit $\hat{y} = b_0 + b_1x$ and make point predictions. Here we add **interval** estimates: a CI for the slope β_1 , a CI for the **mean** response at x_0 , and a **prediction interval** for one individual — and we explain why the PI is always wider.

Inference in simple linear regression

If the linear model is appropriate and residuals are roughly normal with constant spread, we can do interval estimation:

- Confidence interval for the slope `beta1`.
- Confidence interval for the mean response $E(Y \mid X = x_0)$.
- Prediction interval for an individual future Y at x_0 .

The prediction interval is wider than the mean-response confidence interval because it includes both uncertainty in the mean and individual-to-individual variation.

In this section

When residuals are roughly normal, use `confint()` and `predict(..., interval=)` for interval inference.

Example 1 - Study hours and exam score (continued)

```
if (!requireNamespace("tidyverse", quietly = TRUE)) {  
  install.packages("tidyverse", repos = "https://cloud.r-project.org")  
}  
  
library(tidyverse)  
  
set.seed(3501)  
n <- 160  
exam_dat <- tibble(  
  study_hours = runif(n, 1, 15),  
  exam_score = 45 + 3.4 * study_hours + rnorm(n, 0, 8)  
)
```

Train/test split and fit

```
idx_train <- sample(seq_len(nrow(exam_dat)), size = 0.8 * nrow(exam_dat))  
exam_train <- exam_dat[idx_train, ]  
exam_test  <- exam_dat[-idx_train, ]
```

```
fit_exam <- lm(exam_score ~ study_hours, data = exam_train)
summary(fit_exam)
```

Call:

```
lm(formula = exam_score ~ study_hours, data = exam_train)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-16.0920	-4.3112	-0.8295	4.4390	21.3982

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	42.4805	1.3406	31.69	<2e-16 ***
study_hours	3.8436	0.1551	24.78	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.664 on 126 degrees of freedom

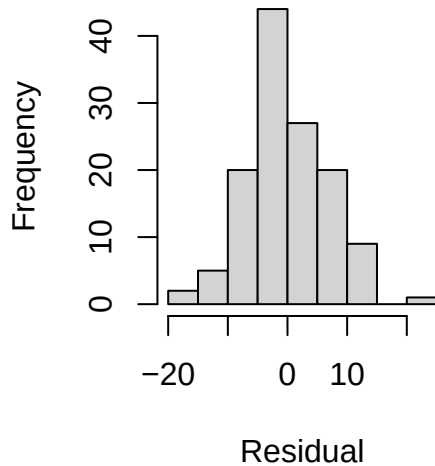
Multiple R-squared: 0.8298, Adjusted R-squared: 0.8284

F-statistic: 614.2 on 1 and 126 DF, p-value: < 2.2e-16

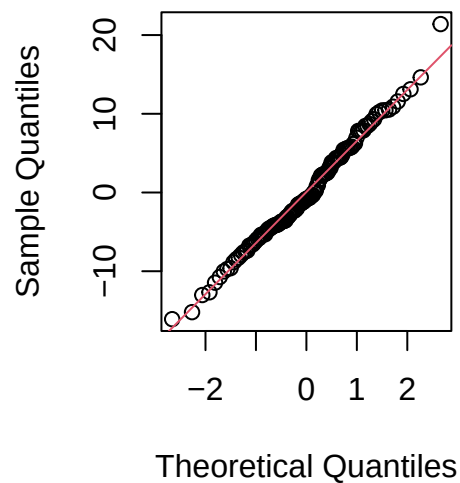
Residual normality check (visual)

```
par(mfrow = c(1, 2))
hist(residuals(fit_exam), main = "Residual histogram", xlab = "Residual")
qqnorm(residuals(fit_exam)); qqline(residuals(fit_exam), col = 2)
```

Residual histogram



Normal Q-Q Plot



```
par(mfrow = c(1, 1))
```

1) CI for slope β_1

```
confint(fit_exam, "study_hours", level = 0.95)
```

```
          2.5 %    97.5 %  
study_hours 3.536713 4.150568
```

Interpretation pattern: we are 95% confident that each additional study hour changes mean exam score by a value in this interval.

2) CI for mean response at x_0

For $x_0 = 10$ study hours:

```
x0 <- tibble(study_hours = 10)  
predict(fit_exam, newdata = x0, interval = "confidence", level = 0.95)
```

```
      fit      lwr      upr  
1 80.91687 79.56421 82.26953
```

Interpretation: 95% CI for the mean exam score among all students who study 10 hours.

3) Prediction interval for an individual response at x_0

```
predict(fit_exam, newdata = x0, interval = "prediction", level = 0.95)
```

```
      fit      lwr      upr  
1 80.91687 67.65902 94.17473
```

Interpretation: 95% interval for a single future student's score if they study 10 hours.

Training vs test performance reminder

```
mse <- function(y, yhat) mean((y - yhat)^2)  
  
train_pred <- predict(fit_exam, newdata = exam_train)  
test_pred  <- predict(fit_exam, newdata = exam_test)  
  
train_mse <- mse(exam_train$exam_score, train_pred)  
test_mse  <- mse(exam_test$exam_score, test_pred)  
  
train_r2 <- 1 - sum((exam_train$exam_score - train_pred)^2) /  
  sum((exam_train$exam_score - mean(exam_train$exam_score))^2)  
test_r2  <- 1 - sum((exam_test$exam_score - test_pred)^2) /  
  sum((exam_test$exam_score - mean(exam_test$exam_score))^2)  
  
tibble(  
  set = c("train", "test"),  
  MSE = c(train_mse, test_mse),  
  R2 = c(train_r2, test_r2)  
)
```

```
# A tibble: 2 x 3  
  set      MSE      R2  
<chr> <dbl> <dbl>
```

```
1 train  43.7 0.830
2 test   66.3 0.767
```

Use test metrics as your main generalization report:

- Test MSE = typical squared prediction error on new cases.
- Test R^2 = proportion of test-set variation explained by the linear model.

Example 2 - Fertilizer dose and plant height

```
set.seed(3502)
plant_dat <- tibble(
  dose = runif(95, 0, 12),
  height_cm = 18 + 2.2 * dose + rnorm(95, 0, 5.3)
)

idx <- sample(seq_len(nrow(plant_dat)), size = 0.8 * nrow(plant_dat))
plant_train <- plant_dat[idx, ]

fit_plant <- lm(height_cm ~ dose, data = plant_train)
summary(fit_plant)
```

Call:

```
lm(formula = height_cm ~ dose, data = plant_train)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-10.7136	-3.3558	0.5898	3.7855	8.9421

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	18.3691	1.3412	13.70	<2e-16 ***
dose	2.1394	0.1706	12.54	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.961 on 74 degrees of freedom

Multiple R-squared: 0.68, Adjusted R-squared: 0.6756

F-statistic: 157.2 on 1 and 74 DF, p-value: < 2.2e-16

```
# CI for slope beta1
```

```
confint(fit_plant, "dose")
```

	2.5 %	97.5 %
dose	1.799457	2.479396

```
# CI for mean response at x0 = 7
```

```
new_x <- tibble(dose = 7)
```

```
predict(fit_plant, newdata = new_x, interval = "confidence")
```

	fit	lwr	upr
1	33.34505	32.2105	34.4796

```
# Prediction interval for an individual at x0 = 7
predict(fit_plant, newdata = new_x, interval = "prediction")
```

```
      fit      lwr      upr
1 33.34505 23.3956 43.2945
```

Example 3 - Exercise minutes and resting heart rate

```
set.seed(3503)
heart_dat <- tibble(
  exercise_min = runif(110, 0, 60),
  resting_hr = 79 - 0.30 * exercise_min + rnorm(110, 0, 4.8)
)

idx <- sample(seq_len(nrow(heart_dat)), size = 0.8 * nrow(heart_dat))
heart_train <- heart_dat[idx, ]

fit_heart <- lm(resting_hr ~ exercise_min, data = heart_train)
summary(fit_heart)
```

Call:

```
lm(formula = resting_hr ~ exercise_min, data = heart_train)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-11.6123  -3.1631   0.3452   2.9383  14.2460
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  80.00362    1.17719   67.962 < 2e-16 ***
exercise_min -0.31807    0.03237  -9.827 1.02e-15 ***
---

```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 4.854 on 86 degrees of freedom

Multiple R-squared: 0.5289, Adjusted R-squared: 0.5235

F-statistic: 96.57 on 1 and 86 DF, p-value: 1.023e-15

```
# CI for slope beta1
confint(fit_heart, "exercise_min")
```

```
      2.5 %      97.5 %
exercise_min -0.3824177 -0.2537277
```

```
# CI for mean resting HR at x0 = 30 min
new_x <- tibble(exercise_min = 30)
predict(fit_heart, newdata = new_x, interval = "confidence")
```

```
      fit      lwr      upr
1 70.46144 69.41853 71.50435
```

```
# Prediction interval for one person at x0 = 30 min
predict(fit_heart, newdata = new_x, interval = "prediction")
```

	fit	lwr	upr
1	70.46144	60.75523	80.16766

Key takeaways

- Slope CI answers: what plausible values for the linear effect **beta1** are supported by data?
 - Mean-response CI answers: where is the average outcome for all units at x_0 ?
 - Prediction interval answers: where might one future individual outcome fall at x_0 ?
 - Keep reporting test MSE and test R^2 for predictive performance on new data. ::: {.callout-tip icon=false} ###
Check yourself — Inference in simple linear regression
1. State the residual assumption needed before computing CIs and prediction intervals in regression.
 2. Output: `confint(fit, 'study_hours')` → [2.44, 3.68]. Interpret this 95% CI for the slope.
 3. At $x_0 = 10$ hours, `predict(fit, interval = 'confidence')` gives [76.1, 80.5]. Interpret.
 4. At $x_0 = 10$ hours, `predict(fit, interval = 'prediction')` gives [62.3, 94.3]. Interpret and compare to the CI above.
 5. In one sentence, explain why prediction intervals are always wider than confidence intervals for the mean response.
- :::

Answers — Inference in simple linear regression

1. Residuals should be approximately **normally distributed** with **constant spread** (homoscedasticity) across all x values.
2. We are 95% confident that each additional hour of study changes the **mean** exam score by between 2.44 and 3.68 points.
3. We are 95% confident the **mean** exam score for all students who study 10 hours is between 76.1 and 80.5 points.
4. We predict with 95% confidence that **one individual** student studying 10 hours will score between 62.3 and 94.3. **Wider** than the mean-response CI because it adds individual-to-individual variation on top of uncertainty in the mean.
5. Predicting **one individual** involves both uncertainty about the mean (like the CI) **plus** the individual's random deviation from the mean.